PATH-CONDITIONED RACE-TREE OUTPUT WITH NORMALIZED CAUSAL COPY SELECTION

Technical field

[0001] Output circuits and processors for large-vocabulary event or sequence prediction.

Background

[0002] A dense output layer scores every vocabulary item. A tree output can reduce decision work but must preserve branch validity and normalized probabilities when combined with a persistent copy memory.

Summary

[0003] A processor uses a training-built vocabulary tree, refines a query after selected branches, and combines the normalized tree law with a causal copy race through a probability gate.

Brief description of the drawings

[0004] FIG. 1 is a block diagram of an implementation. FIG. 2 shows operations of the implementation. Reference signs identify the same functional elements throughout the drawings.

Detailed description

[0005] In FIG. 1, 100 identifies context; 110 identifies tree nodes; 120 identifies child scores; 130 identifies traversal; 140 identifies output; 150 identifies copy store. In FIG. 2, 210 identifies child race, 220 identifies query refine and 230 identifies next node.

[0006] A training-only procedure builds a vocabulary tree with bounded branching factor. Each vocabulary item has one valid leaf path. Node storage 110 records valid child masks, child keys and query-refinement vectors. A child leading to no item is excluded from normalization.

[0007] At node n_k, score engine 120 computes logits $s_{k,j} = q_k^\top e_{k,j} + b_{k,j}$ for valid children. The probability of a token is the product of its conditional branch probabilities. For an observed training target, its path identifies the likelihood factors to score; inference does not obtain an unrevealed target to choose a path.

[0008] After a selected branch j, a representative refinement is $q_{k+1} = \mathrm{LN}(q_k + W_k e_{k,j})$. Branch keys and refinement vectors may be distinct. A sampling decoder follows conditional races; a greedy traversal need not return the globally most probable leaf. Exact maximum-probability search, if requested, requires its own search and cost accounting.

[0009] A causal copy memory stores earlier observed tokens and source features. It assigns normalized weights to admissible past positions and sums weights for positions containing token v, giving $p_{\mathrm{copy}}(v)$. If no copy candidate exists, the implementation sets the copy gate to zero or supplies an explicit null/base policy.

[0010] For gate $g \in [0,1]$, the output law is $p(v) = (1-g)p_{\mathrm{tree}}(v) + g p_{\mathrm{copy}}(v)$. Both component laws are normalized over the same vocabulary. Training evaluates the mixture with a stable log-sum expression. Copy history and document-boundary policy are declared and use only revealed observations.

[0011] For branching factor16 and depth4, one sampled tree path scores at most64 children before refinement work. This is output-path work, not whole-model work, and does not include copy-candidate discovery. FIG. 1 shows context 100, tree 110, child-score 120, traversal/mixing 130, token output 140 and copy store 150. FIG. 2 shows a valid child race 210, query refinement 220 and next-node race 230. A circuit can bank child records by node and issue copy-key scans separately.

Hardware implementation

[0012] A digital implementation may use synthesizable register-transfer logic mapped to an FPGA or to an application-specific integrated circuit. Addressed state uses register banks or SRAM macros; arithmetic uses integer, fixed-point or floating-point datapaths. Ready/valid or request/acknowledge interfaces apply backpressure without accepting a message twice. Synchronous FPGA execution, locally clocked circuits and asynchronous circuits are alternative implementations, each preserving its declared causal ordering and numerical precision.

[0013] Before deployment, a reference trace can check selected addresses, values, timestamps, publication versions and output probabilities against the circuit at the declared numerical precision. Timing quantization, ties, cancellation, overflow and saturation are specified for the implementation. FPGA resource utilization, post-layout timing and measured energy are implementation-dependent quantities; no particular measured hardware result is required by this embodiment.

CLAIMS

1. A method of operating a sequence-output processor, comprising storing a vocabulary tree built from training data with valid-child masks; scoring valid children of a current node using a current query; selecting a child through a conditional race; refining the query according to the selected child before a subsequent tree decision; computing a normalized causal copy distribution from earlier observed items; and combining a tree distribution with the copy distribution through a probability gate.

2. The method of claim 1, wherein a token probability is a product of conditional probabilities along its unique valid leaf path.

3. The method of claim 1, wherein invalid children are excluded from each conditional normalization.

4. The method of claim 1, wherein the copy distribution sums causal position weights for occurrences of the same vocabulary item.

5. The method of claim 1, wherein the copy gate is disabled when no admissible copy candidate is available.

6. The method of claim 1, wherein training evaluates target-path likelihood without providing an unrevealed target to inference-time branch selection.

7. An output-processing apparatus comprising node-key storage, a child race engine, a query-refinement datapath and a causal copy store configured to perform the method of claim 1.

8. A non-transitory computer-readable storage medium storing instructions that cause a processor to perform the method of claim 1.

ABSTRACT

A sequence-output processor uses a training-built vocabulary tree with valid-child masks. Conditional races select branches and refine a query for later decisions. A token law is formed from normalized branch probabilities. A causal memory supplies a separately normalized copy distribution over earlier observed items. A gate mixes the tree and copy distributions. Invalid branches and empty copy sets receive explicit handling. The processor accounts for tree decisions, query refinement and copy discovery separately from the rest of the model.